

Banco de Dados

Arquitetura do banco de dados do Prumo: modelo de relacionamentos, tabelas, formato JSON do quadro, controle de revisões e preparação do PostgreSQL.

Prumo 1.0.0 · PostgreSQL 17 · Django 6.1 · Copyright (©) 2026 Clara Portes

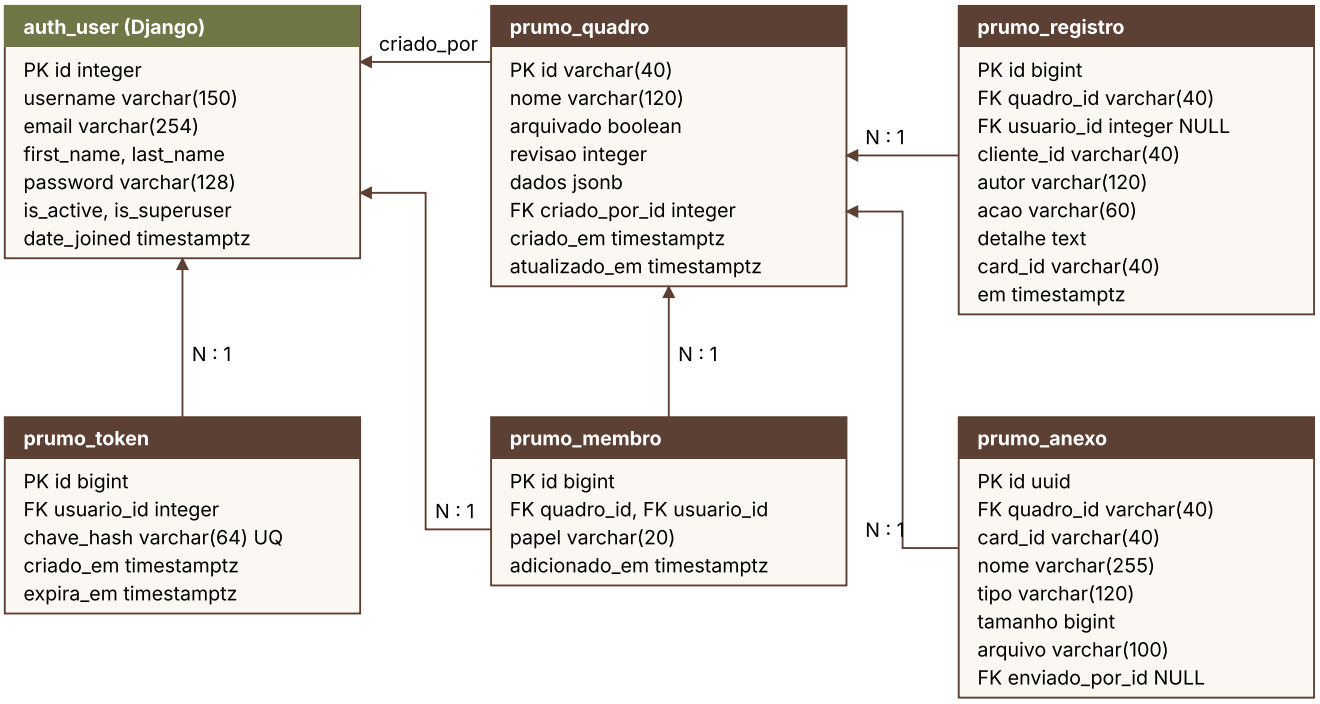
1. Decisões de arquitetura

O Prumo combina duas formas de guardar dados, cada uma onde rende mais:

Dado	Onde fica	Por quê
Conteúdo do quadro (colunas, tarefas, etiquetas, automações, histórico recente)	Uma coluna `jsonb` em `prumo_quadro.dados`	O quadro é lido e salvo inteiro pela interface, que aplica as regras em JavaScript. Um documento por quadro deixa o salvamento atômico e o desfazer/refazer simples.
Quem pode acessar cada quadro	`prumo_membro`	Precisa de chave estrangeira, unicidade e consulta rápida por usuário.
Auditoria permanente	`prumo_registro`	O histórico dentro do JSON guarda só os últimos 500 itens; a tabela guarda tudo, com o usuário autenticado.
Arquivos	`prumo_anexo` + disco	Binários não devem ir para o JSON nem para o banco.
Sessões da API	`prumo_token`	Só o hash SHA-256 do token é guardado.

Para evitar que duas pessoas sobrescrevam o trabalho uma da outra, cada quadro tem uma **revisão**. Todo `PUT` informa a revisão que o navegador conhece; o servidor trava a linha, compara e só grava se forem iguais (controle otimista de concorrência).

2. Diagrama de relacionamentos



Regras de exclusão
Excluir um quadro apaga em cascata membros, registros e anexos. Excluir um usuário apaga seus tokens e participações; nos quadros, registros e anexos o autor vira NULL (SET NULL), preservando a auditoria.

3. Tabelas

prumo_quadro

Campo	Tipo	Restrições	Descrição
id	varchar(40)	PK	Id gerado no navegador (ex.: `qdr_3f9a1c2b7d4e`), o mesmo dentro de `dados.id`
nome	varchar(120)	NOT NULL	Cópia de `dados.nome` para listar sem abrir o JSON
arquivado	boolean	NOT NULL, padrão false	Cópia de `dados.arquivado`
revisao	integer	NOT NULL, CHECK ≥ 0, padrão 1	Soma 1 a cada salvamento aceito
dados	jsonb	NOT NULL	Documento completo do quadro (seção 4)
criado_por_id	integer	FK auth_user, NULL, SET NULL	Quem enviou o quadro
criado_em / atualizado_em	timestamptz	NOT NULL	Automáticos

Índice: `quadro_arquivado_nome_idx (arquivado, nome)` .

prumo_membro

Campo	Tipo	Restrições	Descrição
id	bigint	PK identity	
quadro_id	varchar(40)	FK prumo_quadro, CASCADE	
usuario_id	integer	FK auth_user, CASCADE	
papel	varchar(20)	CHECK em (admin, gerente, membro, visualizador)	Restrição `membro_papel_valido`
adicionado_em	timestamptz	NOT NULL	

Unicidade: `membro_unico_por_quadro (quadro_id, usuario_id)` . Superusuários veem todos os quadros como administradores sem precisar de linha aqui.

prumo_registro

Campo	Tipo	Restrições	Descrição
id	bigint	PK identity	
quadro_id	varchar(40)	FK, CASCADE	

usuario_id	integer	FK, NULL, SET NULL	Usuário do token que salvou
cliente_id	varchar(40)		Id do registro gerado no navegador
autor	varchar(120)	NOT NULL	Nome do usuário autenticado, ou “Automação”
acao	varchar(60)	NOT NULL	criou, moveu, editou, comentou, anexou, aplicou regra...
detalhe	text		Ex.: “Corrigir travamento: To Do → In Progress”
card_id	varchar(40)		Tarefa afetada, quando houver
em	timestamptz	NOT NULL	Momento da ação no navegador

Índices: `registro_quadro_em_idx (quadro_id, em DESC)` e `registro_quadro_card_idx (quadro_id, card_id)`. Unicidade parcial `registro_cliente_unico (quadro_id, cliente_id) WHERE cliente_id <> ''`: se o navegador reenviar o mesmo registro depois de uma falha de rede, ele não duplica.

prumo_anexo

Campo	Tipo	Restrições	Descrição
id	uuid	PK	Também compõe a URL de download
quadro_id	varchar(40)	FK, CASCADE	
card_id	varchar(40)	NOT NULL	Tarefa dona do anexo
nome / tipo	varchar(255) / varchar(120)		Nome original e MIME
tamanho	bigint	CHECK ≥ 0	Bytes
arquivo	varchar(100)	NOT NULL	Caminho relativo a `PRUMO_MEDIA_ROOT`: `{quadro}/{uuid}-{nome}`
enviado_por_id	integer	FK, NULL, SET NULL	
enviado_em	timestamptz	NOT NULL	

Índice: `anexo_quadro_card_idx (quadro_id, card_id)`.

prumo_token

Campo	Tipo	Restrições	Descrição
id	bigint	PK identity	
usuario_id	integer	FK auth_user, CASCADE	
chave_hash	varchar(64)	UNIQUE	SHA-256 do token; o token em si nunca é gravado
criado_em / expira_em	timestamptz	NOT NULL	Validade definida por `PRUMO_TOKEN_HORAS`

As tabelas `auth_user`, `auth_group`, `auth_permission`, `django_content_type` e `django_migrations` são criadas pelo próprio Django.

4. Formato JSON do quadro

O mesmo formato é usado no `localStorage`, na coluna `dados`, na exportação JSON e no arquivo `exemplos/lancamento.json`.

```
{
  "id": "qdr_3f9a1c2b7d4e", "nome": "Sprint 14", "descricao": "", "cor": "#707645",
  "modelo": "scrum", "arquivado": false, "bloquearDependencias": true,
  "criadoEm": "2026-10-01T13:00:00.000Z", "atualizadoEm": "2026-10-21T19:40:12.000Z",
  "colunas": [
    { "id": "col_a1", "nome": "Sprint Backlog", "tipo": "espera",     "limiteWip": 0 },
    { "id": "col_a2", "nome": "Em andamento",   "tipo": "andamento",  "limiteWip": 4 },
    { "id": "col_a3", "nome": "Pronto",         "tipo": "concluido",  "limiteWip": 0 }
  ],
  "etiquetas": [ { "id": "etq_b1", "nome": "Bug", "cor": "#6B1F33" } ],
  "cards": [ {
    "id": "crd_c1", "colunaId": "col_a2", "ordem": 0,
    "titulo": "Corrigir login", "descricao": "...", "responsavel": "Carla Dias",
    "prioridade": "alta", "etiquetas": ["etq_b1"], "inicio": "", "prazo": "2026-10-24",
    "subtarefas":  [ { "id": "sub_1", "texto": "Reproduzir", "feito": true } ],
    "comentarios": [ { "id": "cmt_1", "autor": "Bruno Lima", "texto": "@Carla veja",
"em": "..." } ],
    "anexos":      [ { "id": "...", "nome": "erro.png", "tipo": "image/png", "tamanho":
48213,
                       "url": "/api/v1/anexos/..." } ],
    "dependencias": ["crd_c0"],
    "criadoEm": "...", "iniciadoEm": "...", "concluidoEm": null,
    "movimentos": [ { "de": "col_a1", "para": "col_a2", "em": "..." } ]
  } ],
  "automacoes": [ { "id": "aut_1", "nome": "Review para o Bruno", "ativa": true,
                    "gatilho": { "tipo": "entrar", "colunaId": "col_a2" },
                    "acao":    { "tipo": "atribuir", "valor": "Bruno Lima" } } ],
  "historico": [ { "id": "reg_1", "em": "...", "autor": "Ana Souza", "acao": "moveu",
                   "detalhe": "Corrigir login: Sprint Backlog → Em andamento", "cardId":
"crd_c1" } ],
  "membros": [ { "nome": "Ana Souza", "email": "admin@prumo.dev", "papel": "admin" } ]
}
```

Campo	Valores aceitos
`colunas[].tipo`	`espera`, `andamento`, `concluido`
`cards[].prioridade`	`urgente`, `alta`, `media`, `baixa`
`cards[].prazo`, `inicio`	`AAAA-MM-DD` ou vazio
`automacoes[].gatilho.tipo`	`entrar`, `criar`
`automacoes[].acao.tipo`	`atribuir`, `prioridade`, `etiqueta`, `mover`, `concluir_subtarefas`, `prazo_dias`
Datas com hora	ISO 8601 em UTC

O servidor valida que `id` é texto de até 40 caracteres e que `colunas` e `cards` são listas; o limite do corpo é 5 MB. Para membros, o servidor compara `colunas`, `etiquetas`, `automacoes` e `bloquearDependencias` com a versão gravada e recusa (403) qualquer diferença.

5. Controle de revisões

1. O navegador guarda a revisão que recebeu no `GET` ou no último `PUT`.
2. No `PUT`, o servidor abre uma transação e executa `SELECT ... FOR UPDATE` na linha do quadro.
3. Se a revisão enviada for diferente da gravada, responde 409 com `{revisao, dados}` atuais e não grava nada.
4. Se for igual, grava `dados`, faz `revisao = revisao + 1`, insere os registros novos com `ON CONFLICT DO NOTHING` e confirma.
5. Depois do commit, envia o evento `quadro` pelo channel layer para o grupo `quadro_{id}`.

```
BEGIN;
SELECT revisao FROM prumo_quadro WHERE id = 'qdr_demonstracao' FOR UPDATE;
UPDATE prumo_quadro SET dados = $1, nome = $2, arquivado = $3,
       revisao = revisao + 1, atualizado_em = now()
 WHERE id = 'qdr_demonstracao';
INSERT INTO prumo_registro (...) VALUES (...) ON CONFLICT DO NOTHING;
COMMIT;
```

6. Consultas prontas

Tarefas atrasadas de todos os quadros, direto do JSON:

```
SELECT q.nome AS quadro, c->>'titulo' AS tarefa, c->>'responsavel' AS responsavel,
       (c->>'prazo')::date AS prazo
  FROM prumo_quadro q
 CROSS JOIN LATERAL jsonb_array_elements(q.dados->'cards') c
  JOIN LATERAL jsonb_array_elements(q.dados->'colunas') col ON col->>'id' = c-
>>'colunaId'
 WHERE c->>'prazo' <> '' AND (c->>'prazo')::date < current_date
   AND col->>'tipo' <> 'concluido' AND NOT q.arquivado
 ORDER BY prazo;
```

Lead time médio por quadro, em dias:

```
SELECT q.nome,
       round(avg(extract(epoch FROM ((c->>'concluidoEm')::timestamptz
             - (c->>'criadoEm')::timestamptz)) / 86400)::numeric, 1) AS lead_time_dias
  FROM prumo_quadro q
 CROSS JOIN LATERAL jsonb_array_elements(q.dados->'cards') c
 WHERE c->>'concluidoEm' IS NOT NULL
 GROUP BY q.nome;
```

Quem mais moveu tarefas nos últimos 30 dias:

```
SELECT autor, count(*) AS movimentos
  FROM prumo_registro
 WHERE acao = 'moveu' AND em > now() - interval '30 days'
 GROUP BY autor ORDER BY movimentos DESC;
```

Quadros e papéis de uma pessoa:

```
SELECT q.nome, m.papel
  FROM prumo_membro m JOIN prumo_quadro q ON q.id = m.quadro_id
  JOIN auth_user u ON u.id = m.usuario_id
 WHERE lower(u.email) = 'gerente@prumo.dev';
```

7. Preparando o PostgreSQL

Com Docker nada precisa ser feito: o serviço `postgres` do `docker-compose.yml` cria o banco e o usuário a partir do `.env`. Para um servidor próprio:

```
CREATE ROLE prumo LOGIN PASSWORD 'troque-esta-senha';
CREATE DATABASE prumo OWNER prumo ENCODING 'UTF8' TEMPLATE template0;
\c prumo
ALTER SCHEMA public OWNER TO prumo;
```

```
PRUMO_DB_ENGINE=postgresql
PRUMO_DB_HOST=localhost
PRUMO_DB_PORT=5432
PRUMO_DB_NAME=prumo
PRUMO_DB_USER=prumo
PRUMO_DB_PASSWORD=troque-esta-senha

python manage.py migrate
python manage.py criarusuario --email voce@empresa.com --nome "Seu Nome" --admin
```

Índices extras para muitos quadros

Se você pretende consultar o JSON com frequência (seção 6), crie um índice GIN:

```
CREATE INDEX quadro_dados_gin ON prumo_quadro USING gin (dados jsonb_path_ops);
```

Backup e restauração

```
docker compose exec postgres pg_dump -U prumo -Fc prumo > prumo.dump
docker compose exec -T postgres pg_restore -U prumo -d prumo --clean < prumo.dump
```

Os anexos ficam no volume `anexos` (`/dados/anexos` no contêiner) e devem entrar no backup junto com o banco.

Manutenção

- Tokens vencidos são apagados a cada login do mesmo usuário. Para limpar tudo: `DELETE FROM prumo_token WHERE expira_em < now();`
- O histórico permanente cresce com o uso. Para manter um ano: `DELETE FROM prumo_registro WHERE em < now() - interval '1 year';`
- Com SQLite (desenvolvimento), as mesmas tabelas existem; `jsonb` vira `text` com funções JSON do SQLite.

Copyright (©) 2026 Clara Portes. Distribuído sob a licença MIT.